

TASK 03 — Circuit Breaker & Fallback

Step 1 — Study Circuit Breaker

A **Circuit Breaker** is a resilience pattern used in **microservices** to prevent one failing service from continuously affecting other services.

Order Service —————▶ Payment Service

The Order Service depends on the Payment Service.

If the Payment Service becomes slow or unavailable, the Order Service may keep sending requests. If this continues, threads get blocked, requests time out, and eventually the Order Service can also fail.

The **Circuit Breaker** prevents this situation.

Understand the 3 States

1. CLOSED State

This is the **normal/healthy state**.

Order Service



Circuit Breaker



Payment Service



SUCCESS

The circuit breaker allows requests to go to the Payment Service.

For example:

Request 1 → Payment Service → Success

Request 2 → Payment Service → Success

Request 3 → Payment Service → Success

The circuit remains:

CLOSED

What happens when there are failures:

Suppose the Payment Service starts failing:

Request → Failure

Request → Failure

Request → Failure

Request → Failure

The circuit breaker counts these failures.

If failures cross a configured threshold, for example:

Failure Threshold = 5

then:

CLOSED



Too many failures



OPEN

2. OPEN State

This is the **protection state**.

When the circuit is OPEN, requests are **not sent to the failing service**.

Order Service



Circuit Breaker



X

Payment Service

Instead of calling Payment Service, the circuit breaker immediately rejects/fails the request or returns a fallback response.

For example:

User Request



Order Service



Circuit Breaker



OPEN



Fallback Response

Example fallback:

"Payment service is temporarily unavailable.

Please try again later."

Without a circuit breaker:

1000 requests



Payment Service



Timeout



Timeout



Timeout



Order Service resources exhausted



Order Service fails

This is called a **cascading failure**.

With a circuit breaker:

Payment Service fails



Circuit opens



Requests stop going to Payment Service



Order Service remains protected

3. HALF_OPEN State

The circuit does not remain OPEN forever.

After a configured waiting period, the circuit changes:

OPEN



Wait



HALF_OPEN

Now the circuit breaker sends a **small/test request** to check whether the Payment Service has recovered.

For example:

Circuit = HALF_OPEN

Test Request



Payment Service



Success

If the test request succeeds:

HALF_OPEN



Success



CLOSED

Normal traffic can resume.

If the test request fails:

HALF_OPEN



Failure



OPEN

The circuit protects the application again.

Step 2 — Understand the flow

Healthy Service

Initially the service is working correctly.

Order Service



Circuit Breaker



Payment Service



Success

Circuit state:

CLOSED

Failures Increase

Now the Payment Service starts having problems.

For example:

Request 1 → Failure

Request 2 → Failure

Request 3 → Failure

Request 4 → Failure

Request 5 → Failure

The circuit breaker monitors these failures.

Suppose configuration is:

Failure threshold = 5

After reaching the threshold:

CLOSED → OPEN

Circuit Becomes OPEN

Now requests are blocked.

Client



Order Service



Circuit Breaker



OPEN



Request rejected / fallback

The failing Payment Service is no longer continuously called.

This gives the Payment Service time to recover and protects the calling service.

Wait

The circuit stays OPEN for a configured period.

For example:

Wait Duration = 10 seconds

During these 10 seconds:

OPEN



No normal requests to Payment Service

After the waiting period:

OPEN → HALF_OPEN

HALF_OPEN

The circuit allows a **test request**.

Circuit Breaker



Test Request



Payment Service

The purpose is to answer one question:

Has the Payment Service recovered

Test Request Succeeds

If the test request succeeds:

HALF_OPEN



SUCCESS



CLOSED

Normal requests are allowed again.

Client



Order Service



Circuit Breaker



Payment Service



Success

Test Request Fails

If the test request fails:

HALF_OPEN



FAILURE



OPEN

The circuit opens again.

Payment Service still unhealthy



Circuit OPEN



Requests protected

After another waiting period, the circuit can try HALF_OPEN again.

Step 3 — Add Circuit Breaker Dependency

Now we implement it in Spring Boot.

We will use: **Resilience4j**

It provides:

- Circuit Breaker

- Retry

- Rate Limiter

- Bulkhead

- Time Limiter

For this implementation, we mainly use: Circuit Breaker

Maven Dependency

Open your **Order Service** pom.xml.

```
<dependency>  
  <groupId>io.github.resilience4j</groupId>  
  <artifactId>resilience4j-spring-boot3</artifactId>  
  <version>2.3.0</version>  
</dependency>
```

You can also add Spring Boot Actuator for monitoring:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

So the relevant dependencies become:

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>io.github.resilience4j</groupId>  
    <artifactId>resilience4j-spring-boot3</artifactId>  
    <version>2.3.0</version>  
  </dependency>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
  </dependency>  
</dependencies>
```

If your project already uses Spring Cloud Circuit Breaker or OpenFeign, the dependency arrangement can be different. The code below uses Resilience4j's direct Spring Boot integration.

Step 4 — Protect User Service call

We need to apply Circuit Breaker to the method that calls User Service.

Example:

```
public User getUserById(Long userId)
```

This method calls: User Service

We add:

```
@CircuitBreaker
```

Create User DTO

Inside Order Service:

```
src/main/java/com/example/order/dto/UserResponse.java
```

Create:

```
package com.example.order.dto;

public class UserResponse {

    private Long id;
    private String name;
    private String email;
    public UserResponse() {
    }

    public UserResponse(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
```

```

        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}

```

Create User Service Client

Create:

UserClient.java

```

package com.example.order.client;

import com.example.order.dto.UserResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
public class UserClient {
    private final RestClient restClient;

    public UserClient(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("http://localhost:8081")

```

```

        .build();
    }

    public UserResponse getUserById(Long userId) {
        return restClient
            .get()
            .uri("/users/{id}", userId)
            .retrieve()
            .body(UserResponse.class);
    }
}

```

This code makes:

GET <http://localhost:8081/users/{id}>

Create RestClient Configuration

Create:

RestClientConfig.java

```

package com.example.order.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

    @Bean
    public RestClient.Builder restClientBuilder() {
        return RestClient.builder();
    }
}

```

Add Circuit Breaker

Now create an Order Service class.

```
package com.example.order.service;

import com.example.order.client.UserClient;
import com.example.order.dto.UserResponse;
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final UserClient userClient;

    public OrderService(UserClient userClient) {
        this.userClient = userClient;
    }

    @CircuitBreaker(
        name = "userService",
        fallbackMethod = "userServiceFallback"
    )
    public UserResponse getUser(Long userId) {
        return userClient.getUserById(userId);
    }

    public UserResponse userServiceFallback(
        Long userId,
        Throwable throwable) {
        System.out.println(
            "User Service unavailable: "
            + throwable.getMessage()
        );
        return new UserResponse(
            userId,
            "Fallback User",
        );
    }
}
```

```
        "unavailable@example.com"  
    );  
}  
}
```

The important part is:

```
@CircuitBreaker(  
    name = "userService",  
    fallbackMethod = "userServiceFallback"  
)
```

What does name = "userService" mean

This:

```
name = "userService"
```

is the name of our Circuit Breaker instance.

We will use the same name in:

```
application.properties
```

For example:

```
resilience4j.circuitbreaker.instances.userService.failure-rate-threshold=50
```

Notice:

```
userService
```

must match.

What is fallbackMethod

This: `fallbackMethod = "userServiceFallback"`

means:

If the protected User Service call fails or the Circuit Breaker is OPEN, execute `userServiceFallback()`.

Normal method: `getUser()`

Fallback: `userServiceFallback()`

Flow:

getUser()



Circuit Breaker

|— Success → return User

└— Failure → fallback

Important Fallback Rule

The fallback method should normally have:

same original parameters

+

Throwable

Original:

```
public UserResponse getUser(Long userId)
```

Fallback:

```
public UserResponse userServiceFallback(  
    Long userId,  
    Throwable throwable)
```

So:

Original parameters:

Long userId

Fallback parameters:

Long userId + Throwable

Step 5 — Configure Thresholds

Failure Threshold

The failure threshold determines **when the circuit should OPEN**.

For example:

Failure Rate = 50%

Suppose we have 10 requests:

Request 1 → Success

Request 2 → Failure

Request 3 → Failure

Request 4 → Success

Request 5 → Failure

Request 6 → Failure

Request 7 → Success

Request 8 → Failure

Request 9 → Success

Request 10 → Failure

There are 6 failures out of 10.

Failure rate = 60%

If our threshold is:

50%

then:

60% > 50%

So the Circuit Breaker can change:

CLOSED → OPEN

Failure threshold = How many failures are acceptable before protecting the service.

Sliding Window

The sliding window determines **how many recent calls the Circuit Breaker examines.**

For example:

Sliding Window Size = 10

The Circuit Breaker looks at the latest 10 calls.

Imagine:

Old calls

↓

[1][2][3][4][5][6][7][8][9][10]

A new request arrives:

[2][3][4][5][6][7][8][9][10][11]

Request 1 is removed and Request 11 is added.

That's why it's called a **sliding window**.

Sliding window = Number of recent requests used to calculate failure rate.

Wait Duration

When the circuit becomes OPEN, we don't immediately send requests again.

We wait for some time.

For example:

Wait Duration = 10 seconds

Flow:

CLOSED



Failures increase



OPEN



Wait 10 seconds



HALF_OPEN

During the 10 seconds, normal calls are blocked.

Why wait?

Because the User Service needs time to recover.

If we immediately start sending requests again:

User Service DOWN



Request



Failure



Request



Failure



Request



Failure

we may continue putting pressure on the failed service.

Wait duration = How long the circuit stays OPEN before checking the service again.

Permitted Test Calls

After the wait duration, the Circuit Breaker changes:

OPEN → HALF_OPEN

It allows a limited number of test requests.

Example: Permitted Test Calls = 3

Then:

HALF_OPEN



Test Request 1

Test Request 2

Test Request 3

Suppose all three succeed:

Test 1 → Success

Test 2 → Success

Test 3 → Success



HALF_OPEN → CLOSED

Normal traffic resumes.

If the test calls fail:

Test 1 → Failure

Test 2 → Failure

Test 3 → Failure



HALF_OPEN → OPEN

The circuit opens again.

Step 6 — Create fallback

A **fallback** is an alternative response that the application provides when the dependent service cannot be reached.

For example:

Order Service



Circuit Breaker



User Service

X

unavailable



FALLBACK

Instead of allowing the application to crash or wait indefinitely, Order Service returns a controlled response.

Why do we need a fallback:

Suppose the user requests:

GET /orders/501

Order Service needs User Service to get user information.

But User Service is down.

Without fallback:

Client



Order Service



User Service

X

Timeout



500 Internal Server Error

With fallback:

Client



Order Service



Circuit Breaker



User Service unavailable



Fallback



Controlled response

Example:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "user": null,  
  "message": "User service temporarily unavailable"  
}
```

The important part is:

```
"user": null
```

This tells the client:

We have the order information, but we couldn't retrieve the user information because User Service is temporarily unavailable.

Fallback Code

A better approach is to create a dedicated response class.

OrderResponse.java

```
package com.example.order.dto;
```

```
public class OrderResponse {  
    private Long orderId;  
    private String product;  
    private UserResponse user;  
    private String message;  
    public OrderResponse() {  
    }  
    public OrderResponse(  
        Long orderId,  
        String product,  
        UserResponse user,  
        String message) {  
        this.orderId = orderId;  
        this.product = product;  
        this.user = user;  
        this.message = message;  
    }  
    public Long getOrderId() {  
        return orderId;  
    }  
}
```

```

public void setOrderId(Long orderId) {
    this.orderId = orderId;
}
public String getProduct() {
    return product;
}
public void setProduct(String product) {
    this.product = product;
}
public UserResponse getUser() {
    return user;
}
public void setUser(UserResponse user) {
    this.user = user;
}
public String getMessage() {
    return message;
}
public void setMessage(String message) {
    this.message = message;
}
}

```

Circuit Breaker Service

Now protect the User Service call.

```

package com.example.order.service;

import com.example.order.client.UserClient;
import com.example.order.dto.OrderResponse;
import com.example.order.dto.UserResponse;

```

```

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.stereotype.Service;

@Service
public class OrderService {
    private final UserClient userClient;

    public OrderService(UserClient userClient) {
        this.userClient = userClient;
    }

    @CircuitBreaker(
        name = "userService",
        fallbackMethod = "userServiceFallback"
    )
    public OrderResponse getOrder(Long orderId, Long userId) {
        UserResponse user =
            userClient.getUserById(userId);
        return new OrderResponse(
            orderId,
            "Laptop",
            user,
            "Order retrieved successfully"
        );
    }

    public OrderResponse userServiceFallback(
        Long orderId,
        Long userId,
        Throwable throwable) {
        return new OrderResponse(
            orderId,

```

```

        "Laptop",
        null,
        "User service temporarily unavailable"
    );
}
}

```

Notice the fallback method:

```

public OrderResponse userServiceFallback(
    Long orderId,
    Long userId,
    Throwable throwable)

```

It contains the original parameters:

orderId

userId

plus:

Throwable

Step 7 — Test Healthy Service

This means we first test the system when everything is healthy.

Start:

User Service

Order Service

For example:

User Service → 8081

Order Service → 8082

Request

Use Postman:

GET <http://localhost:8082/orders/501/user/101>

The flow is:

Postman



Order Service :8082



Circuit Breaker



User Service :8081



Success

Expected response:

```
{
  "orderId": 501,
  "product": "Laptop",
  "user": {
    "id": 101,
    "name": "Madhav",
    "email": "madhav@example.com"
  },
  "message": "Order retrieved successfully"
}
```

The Circuit Breaker remains:

CLOSED

Step 8 — Stop User Service

Now we intentionally stop User Service.

Before stopping:

Order Service



Circuit Breaker

CLOSED



User Service

RUNNING

Stop User Service.

Now:

Order Service



Circuit Breaker



User Service

X

DOWN

Generate Multiple Requests

Send the same request multiple times:

GET <http://localhost:8082/orders/501/user/101>

Example:

Request 1 → Failure

Request 2 → Failure

Request 3 → Failure

Request 4 → Failure

Request 5 → Failure

The Circuit Breaker records these failures.

If the configured conditions are reached:

CLOSED



Too many failures



OPEN

What happens when OPEN:

Initially, each request may actually try to reach User Service:

Request



Circuit Breaker CLOSED



User Service

X

Failure

Once the circuit opens:

Request



Circuit Breaker OPEN



Do NOT call User Service



Fallback

So the fallback becomes faster because the application doesn't repeatedly wait for a failed downstream service.

Expected Fallback Response

You should receive something like:

```
{  
  "orderId": 501,  
  "product": "Laptop",
```

```
"user": null,  
"message": "User service temporarily unavailable"  
}
```

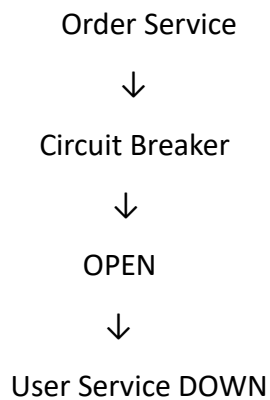
This proves that:

1. User Service is unavailable.
2. Circuit Breaker detected failures.
3. Circuit Breaker opened.
4. Fallback was executed.
5. Order Service remained available.

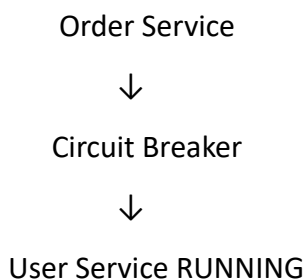
Step 9 — Restart User Service

Now restart User Service.

Before restarting:



Start User Service again:



However, the Circuit Breaker may **not immediately become CLOSED**.

It first waits for the configured duration.

For example: `resilience4j.circuitbreaker.instances.userService.wait-duration-in-open-state=10s`

So:

OPEN



Wait 10 seconds



HALF_OPEN

HALF_OPEN Testing

Once HALF_OPEN, the Circuit Breaker permits a limited number of test requests.

Suppose:

`resilience4j.circuitbreaker.instances.userService.permitted-number-of-calls-in-half-open-state=3`

Then:

HALF_OPEN



Test Request 1



Test Request 2



Test Request 3

If User Service is healthy:

Test 1 → Success

Test 2 → Success

Test 3 → Success

Then:

HALF_OPEN



Success



CLOSED

Normal traffic resumes.

If User Service is Still Down

Suppose you restart incorrectly or User Service still has a problem.

Then:

HALF_OPEN



Test Request



Failure

The circuit goes back to:

OPEN

So:

OPEN



Wait



HALF_OPEN



Failure

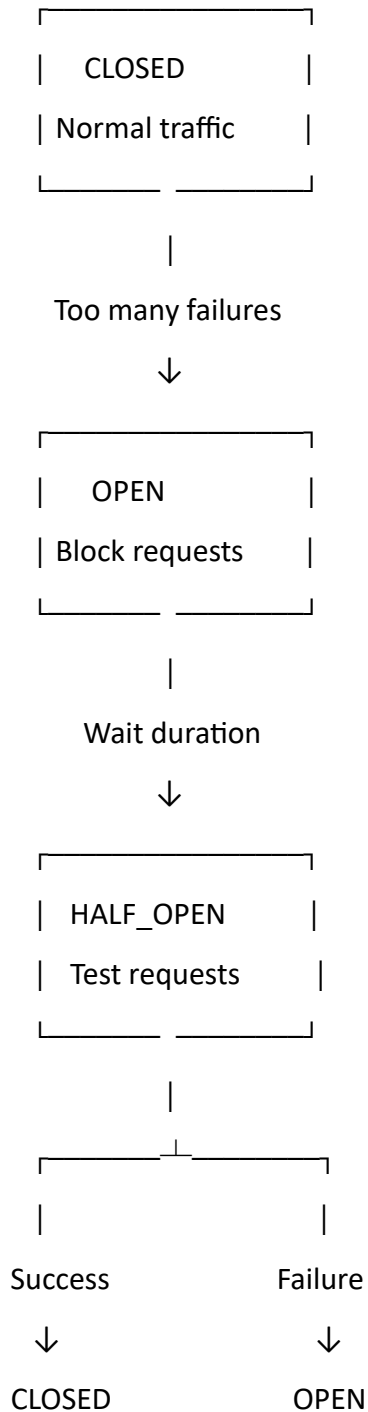


OPEN

The system continues protecting Order Service.

Step 10 — Document

Your documentation should contain the following diagram.



Complete End-to-End Scenario

Let's go through the complete test from beginning to end.

Phase 1 — Both services healthy

Order Service



Circuit Breaker

CLOSED



User Service



SUCCESS

Response:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "user": {  
    "id": 101,  
    "name": "Madhav"  
  },  
  "message": "Order retrieved successfully"  
}
```

Phase 2 — User Service fails

Stop User Service.

Order Service



Circuit Breaker



User Service

X

Requests fail.

Request 1 → Failure

Request 2 → Failure

Request 3 → Failure

Request 4 → Failure

Request 5 → Failure

Circuit opens:

CLOSED → OPEN

Phase 3 — Fallback

Now requests are handled by the fallback.

Order Service



Circuit Breaker

OPEN



Fallback

Response:

```
{  
  "orderId": 501,  
  "product": "Laptop",  
  "user": null,  
  "message": "User service temporarily unavailable"  
}
```

Phase 4 — Wait

Circuit remains:

OPEN

for:

10 seconds

Then: OPEN → HALF_OPEN

Phase 5 — User Service recovers

Restart User Service.

Now test calls are made.

HALF_OPEN



Test call



User Service



SUCCESS

After successful test calls:

HALF_OPEN → CLOSED

Phase 6 — Normal operation resumes

Now:

Order Service



Circuit Breaker

CLOSED



User Service



SUCCESS

Everything is working normally again.